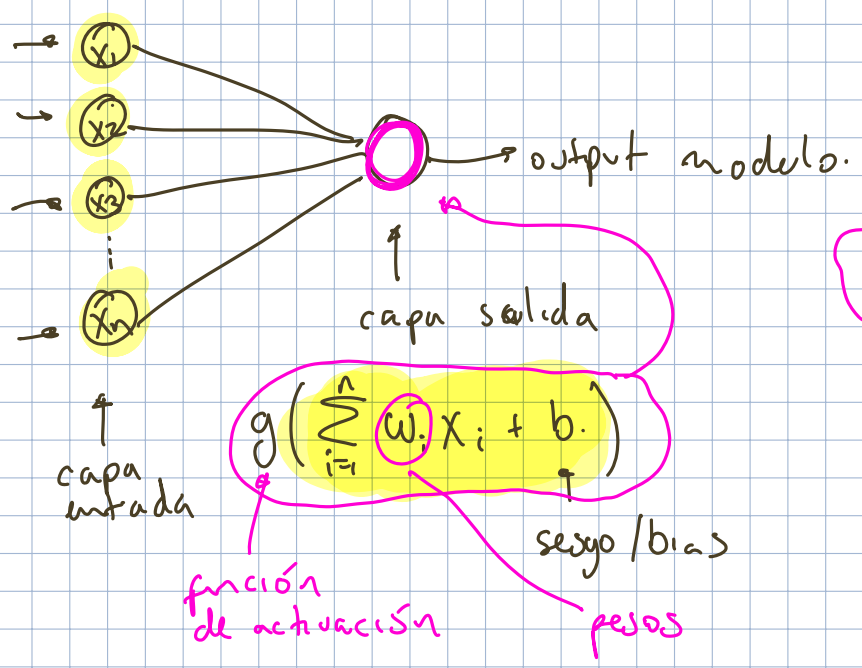
Redes neuronales:

Perceptrón: red neuronal de una capa:

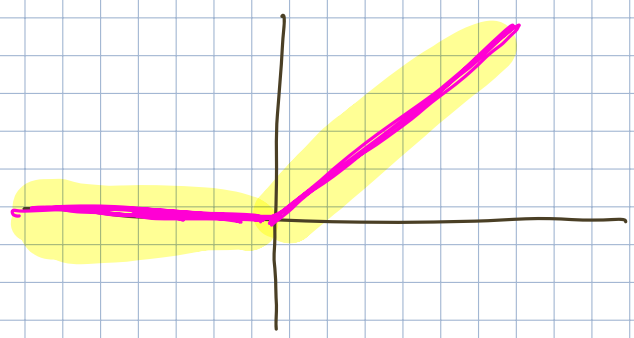
Dataset: n features, y outcome



idea: estimar b, w_i

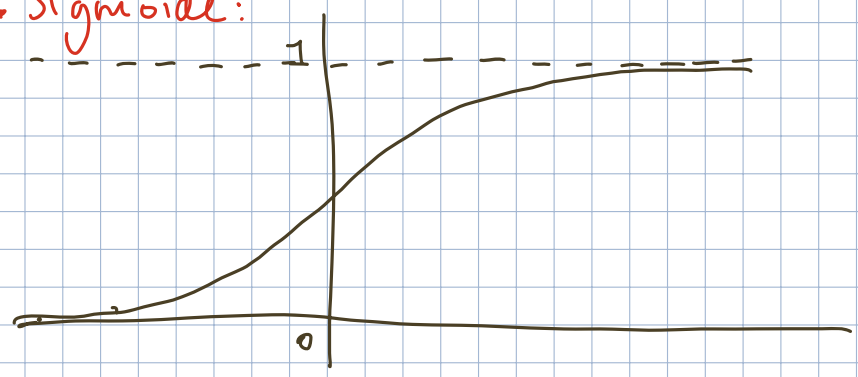
Funciones de activación:

ReLU: rectified linear unit:



$$\mathrm{ReLU}(x) = \begin{cases} x & \text{si } x \geq 0 \\ 0 & \text{si } x < 0 \end{cases}$$

- Sigmoide:

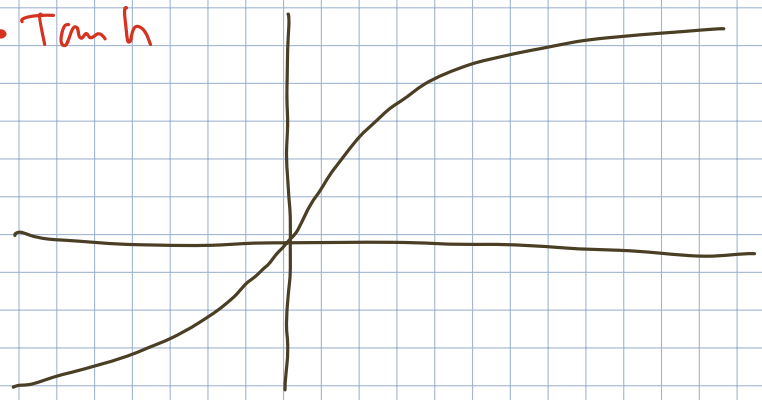


muy usada en problemas de clasificación binaria

$$f(x) = \frac{1}{1+e^{-x}}$$

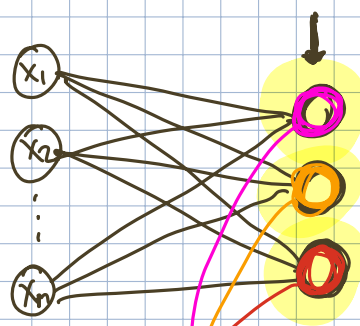
multiclase: softmax

• Tanh



$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

clasificación multiclase: supongamos que tenemos 3 clases:



cada uno de estos outputs es una probabilidad

0.25
0.5
0.25

$$g\Big(\overbrace{\sum_{i=1}^{n} w_i^1 x_i + b^1}^{z_1}\Big)$$

$$g\Big(\overbrace{\sum_{i=1}^{n} w_i^2 x_i + b^2}^{z_2}\Big)$$

$$g\Big(\overbrace{\sum_{i=1}^{n} w_i^3 x_i + b^3}^{z_3}\Big)$$

Ahora debemos estimar $w_i^1, w_i^2, w_i^3, b^1, b^2, b^3$

$i = 1, \dots, n$

función de activación softmax:

$$\begin{pmatrix} \dfrac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}} \\[2ex] \dfrac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}} \\[2ex] \dfrac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}} \end{pmatrix}$$

$$\sum_{i=1}^{n} w_i^1 x_i + b^1 = \begin{pmatrix} 1 & x_1 & x_2 & \cdots & x_n \end{pmatrix} \begin{pmatrix} b^1 \\ w_1^1 \\ w_2^1 \\ \vdots \\ w_n^1 \end{pmatrix} = x^t w^1$$

$$\sum_{i=1}^{n} w_i^2 x_i + b^2 = \begin{pmatrix} 1 & x_1 & \cdots & x_n \end{pmatrix} \begin{pmatrix} b^2 \\ w_1^2 \\ w_2^2 \\ \vdots \\ w_n^2 \end{pmatrix} = X^t w^2$$

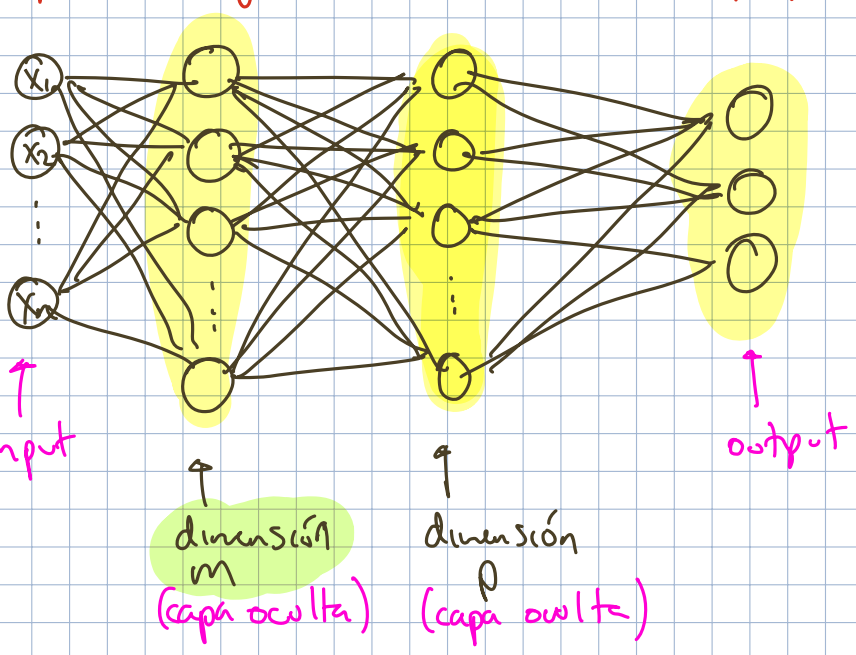
$$\vdots$$

$$g\left(\begin{pmatrix} 1 & x_1 & x_2 & \cdots & x_n \end{pmatrix} \begin{pmatrix} b^1 & b^2 & b^3 \\ w_1^1 & w_1^2 & w_1^3 \\ w_2^1 & w_2^2 & w_2^3 \\ \vdots & \vdots & \vdots \\ w_n^1 & w_n^2 & w_n^3 \end{pmatrix} \right) = g(X^t W)$$

$g(X^t W)$ — output de la red neuronal:

$X^t : 1 \times n$
$W : n \times 3$ $\Rightarrow$ $X^t W : 1 \times 3$

Deep learning: redes neuronales profundas: MLP: multi layer perceptron



densa: todos los outputs de una capa entran como inputs a la capa siguiente.

$$g'\left(\sum_{i=1}^{n} w_i^1 x_i + b^1 \right)$$

$$\vdots$$

$$g'\left(\sum_{i=1}^{n} w_i^m x_i + b^m \right)$$

matriz de pesos de capa 1

$a^{(0)} = x$

$z^{(1)} = a^{(0)t} \cdot W^{(1)}$

$a^{(1)} = g^{1}(z^{(1)})$ ← output de la capa 1

$z^{(2)} = a^{(1)t} W^{(2)}$

$a^{(2)} = g^{2}(z^{(2)})$ ← output de la capa 2

funciones de activación de capas 1 y 2 pueden ser potencialmente distintas

$z^{(3)} = a^{(2)t} W^{(3)}$

$a^{(3)} = g^{3}(z^{(3)}) = \hat{y}$

MQCNN

2 etapas:
- feed forward
- back propagation

} cada pasada hacia adelante y hacia atrás es una "época" – "epoch"

Dataset de entrenamiento:

$(x_1, y_1), (x_2, y_2), \dots (x_S, y_S)$ → tengo S puntos

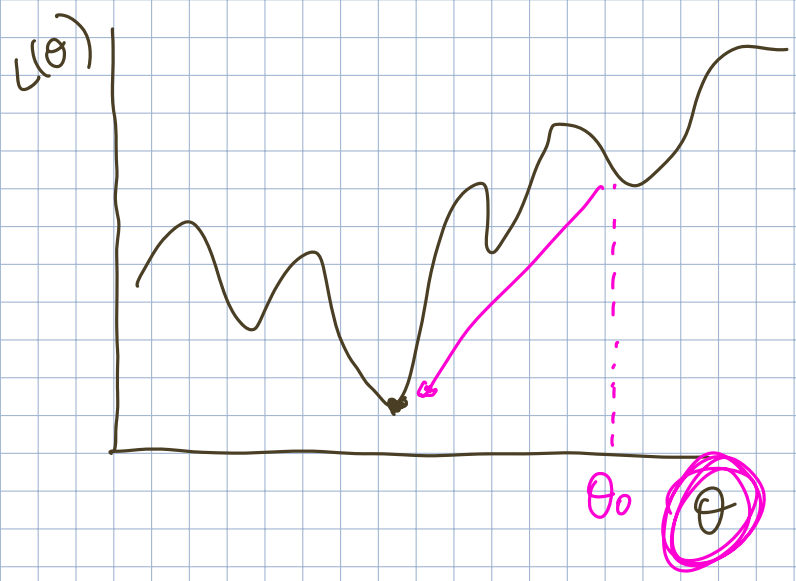
x_1 vector n-dimensional

Inicializamos la red neuronal con pesos aleatorios:
$W^{(1)}, W^{(2)}, W^{(3)}$ con números aleatorios.

feed forward: pasar cada uno de los $s = 1, \dots, S$ datos por la red neuronal con pesos aleatorios para obtener $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_S$

función de pérdida:

- Regresión: $\sum_{i=1}^{S} (y_i - \hat{y}_i)^2$ → MSE
- Clasificación: $\sum_{i=1}^{S} y_i \ln(y_i) + (1-y_i) \ln(1-y_i)$ → binary cross entropy



Stochastic gradient descent:

$$\theta^{(n)} = \theta^{(n-1)} - \rho \frac{\partial L}{\partial \theta}$$

learning rate

Me interesa: $\frac{\partial L}{\partial W^{(3)}}$, $\frac{\partial L}{\partial W^{(2)}}$, $\frac{\partial L}{\partial W^{(1)}}$

$$z^{(\ell)} = a^{(\ell-1)} W^{(\ell)} \qquad a^{(\ell)} = g^{(\ell)}(z^{(\ell)})$$

$$\frac{\partial L}{\partial W^{(\ell)}} = \underbrace{\frac{\partial L}{\partial z^{(\ell)}}}_{\delta^{(\ell)}} \cdot \underbrace{\frac{\partial z^{(\ell)}}{\partial W^{(\ell)}}}_{= a^{(\ell-1)}}$$

$$z^{(L)} = \hat{y}$$

$$\delta^{(L)} = \frac{\partial L}{\partial z^{(L)}} = \frac{\partial L}{\partial \hat{y}} \cdot g^{(L)\prime}(z^{(L)})$$

Para $\ell = L-1, L-2, \ldots, 1$:

$$\delta^{(\ell)} = \left(\delta^{(\ell+1)} W^{(\ell+1)T}\right) \cdot g^{(\ell)\prime}(z^{(\ell)})$$

Regularización:

Regresión: $\sum_{i=1}^{S} (\hat{y}_i - y_i)^2 + \lambda \sum_j |W_j| \longrightarrow$ reg $\underline{L1}$

$+ \lambda \sum_j W_j^2 \longrightarrow$ reg $\underline{L2}$